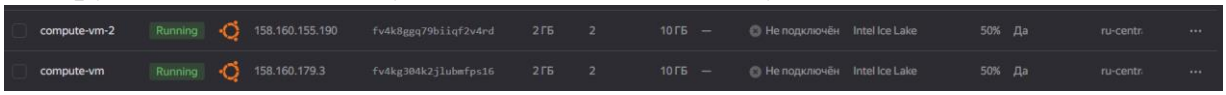
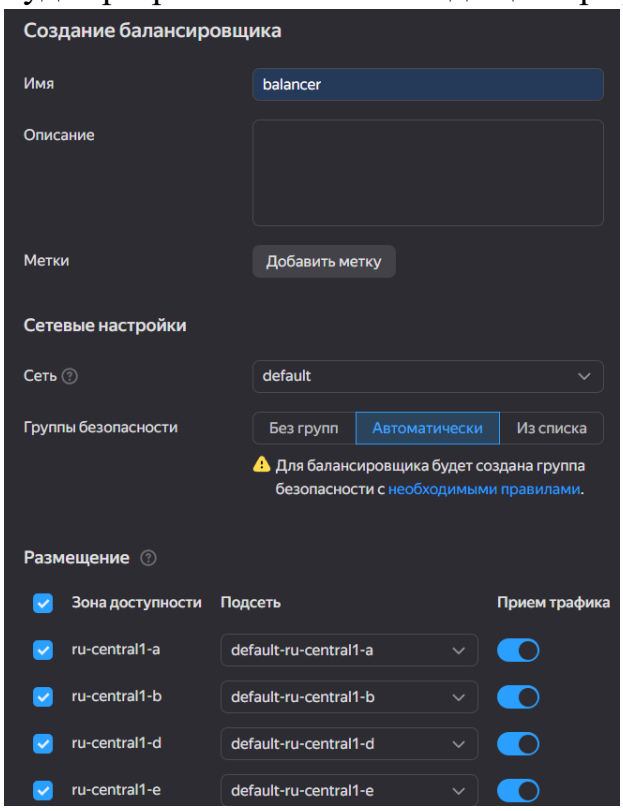
Cloud#3

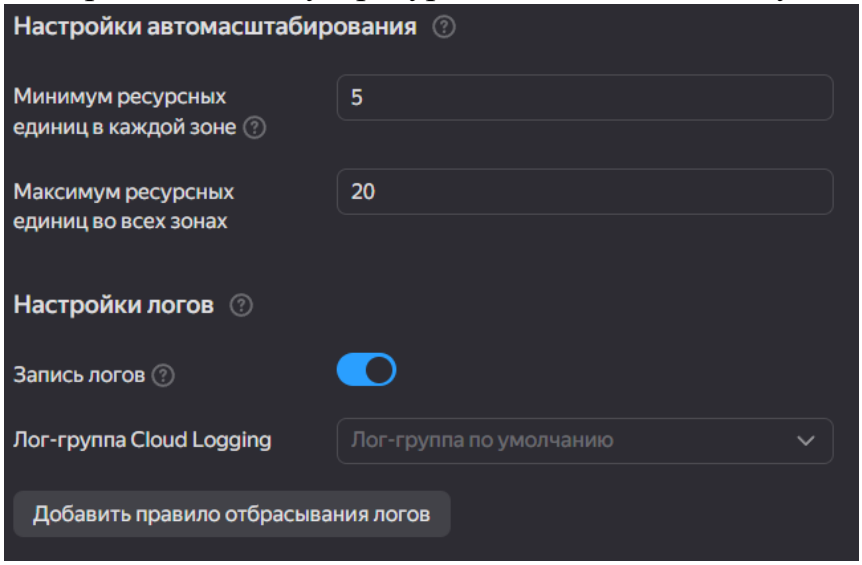
1. Для создания балансировщика необходимо сначала создать целевую группу, в которую добавляем нашу ВМ и создам еще одну ВМ:



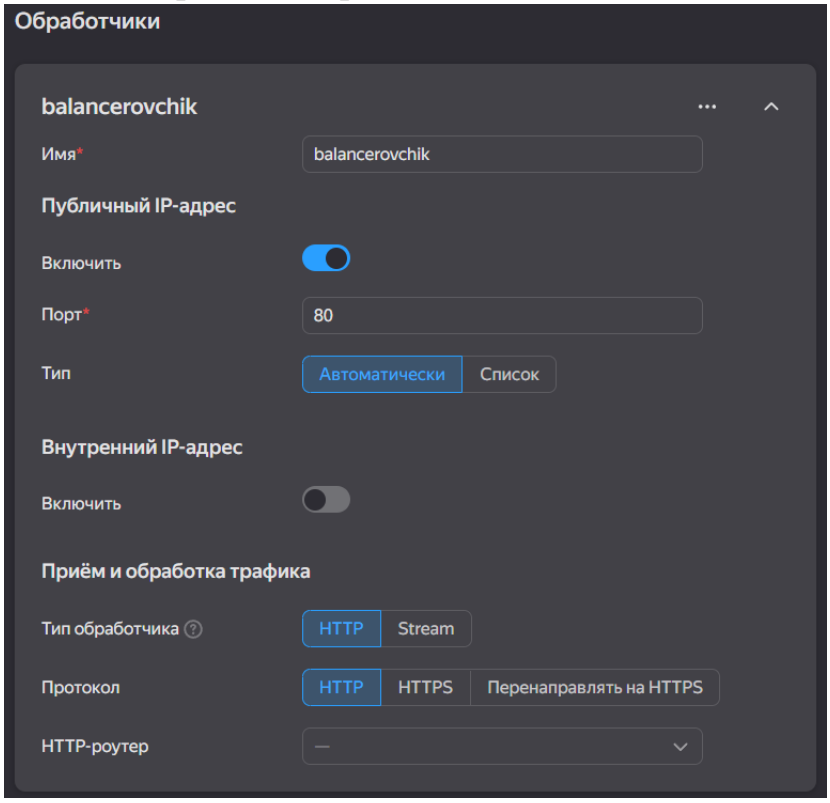
2. Теперь перехожу в раздел Application Load Balancer и нажимаю создать. Группу безопасности выставляю Автоматически, тогда он создаст группу безопасности, разрешающая любой входящий трафик на порте 80 и TCP-трафик для проверки состояния узлов балансировщика на порте 30080. При этом для балансировщика будет разрешен любой исходящий трафик.



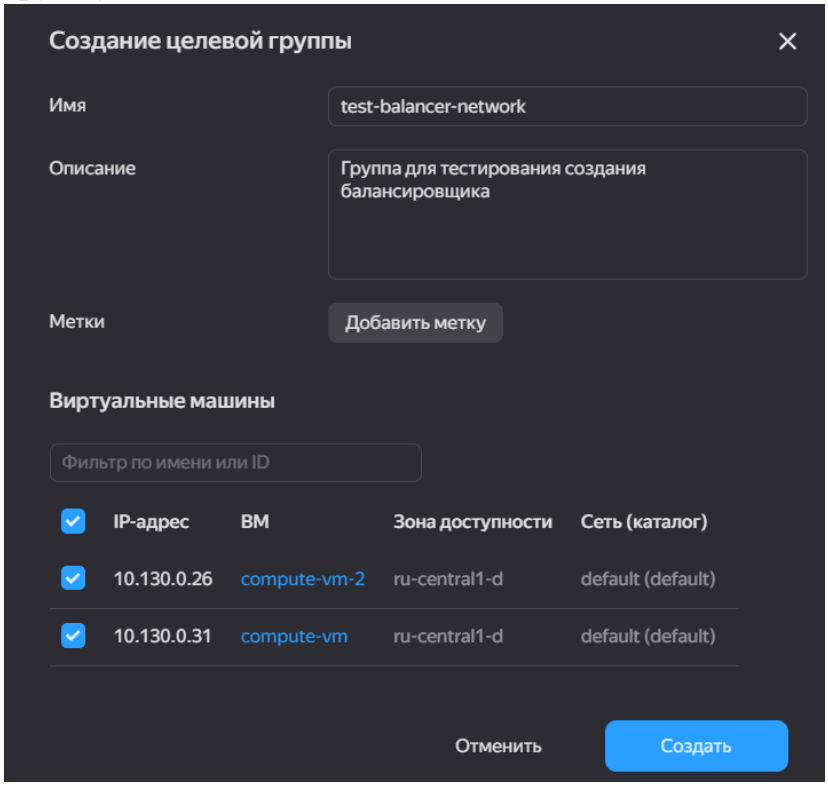
Для автомасштабирования устанавливаю следующие критерии. 20 стоит так как идет расчет минимум ресурсных единиц в зоне, умноженный на количество зон.



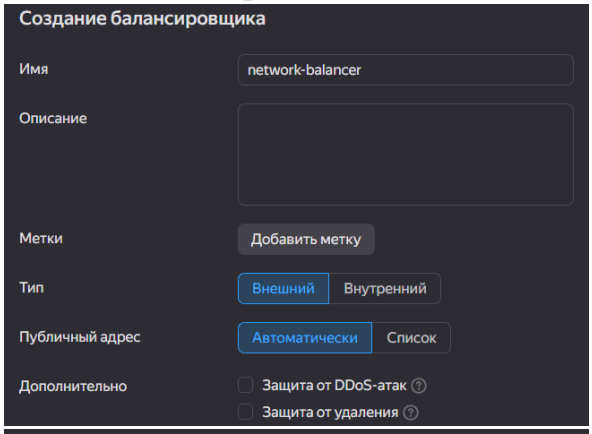
3. Далее настраиваю обработчик

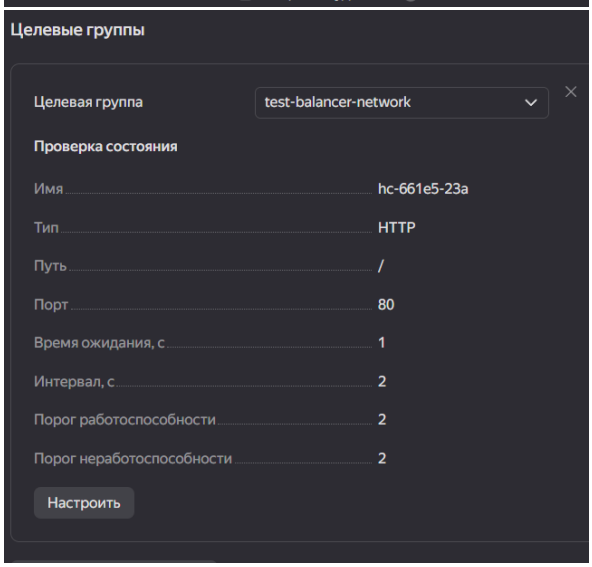


4. Также необходимо настроить целевую группу, в которой будут мои ВМ, чтобы затем настроить сетевой балансировщик, чтобы распределялась нагрузка (либо оставить только первый вариант балансировщика). Для этого нахожу Network Load Balancer и перехожу к созданию, там можно указать или создать целевую группу:

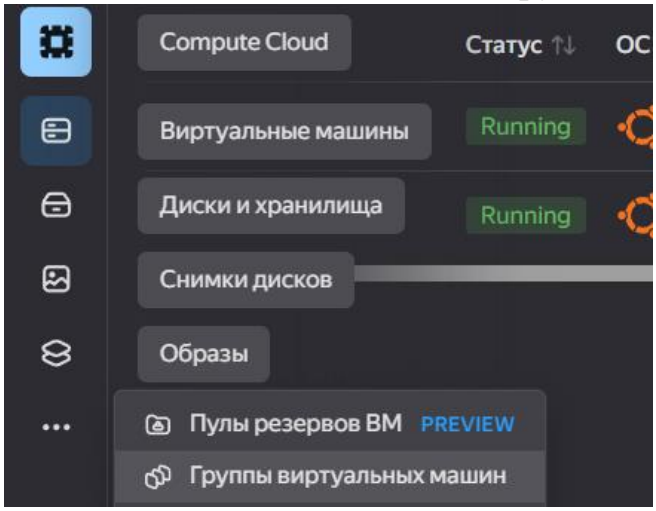


5. Основные настройки создания сетевого балансировщика:

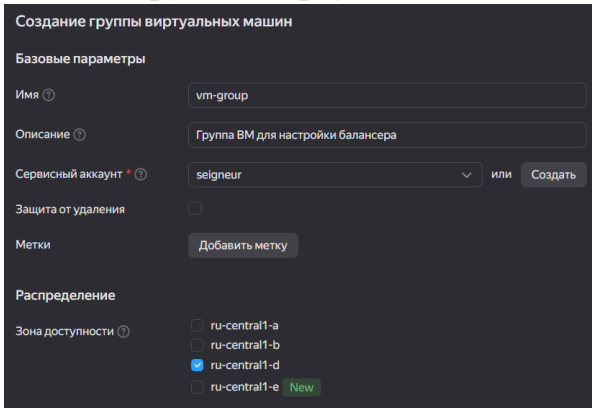




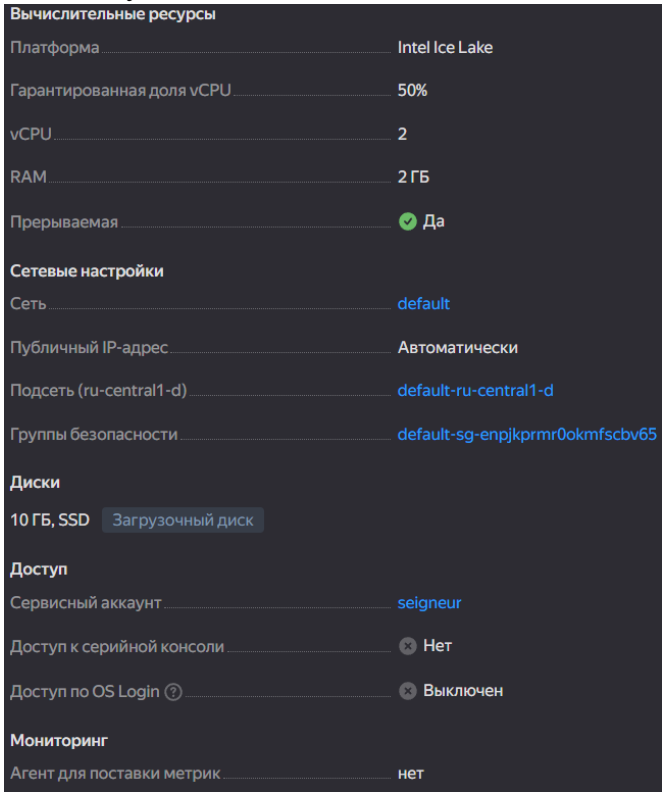
6. Как это выглядит если создавать через CLI:
   yc load-balancer network-load-balancer create \
     --name network-balancer \
     --type external \
     --target-group target-group-id=enpttqgup16mbh2pr8n5,healthcheck-name=hc-661e5-23a,healthcheck-interval=2,healthcheck-timeout=1,healthcheck-unhealthythreshold=2,healthcheck-healthythreshold=2,healthcheck-http-port=80,healthcheck-http-path=/
7. Итак, как оказалось можно было изначально просто создать группу ВМ, где автоматически пометить создания балансировщика. Для этого заходим в пункт с нашими ВМ и слева находим «Группа ВМ»:



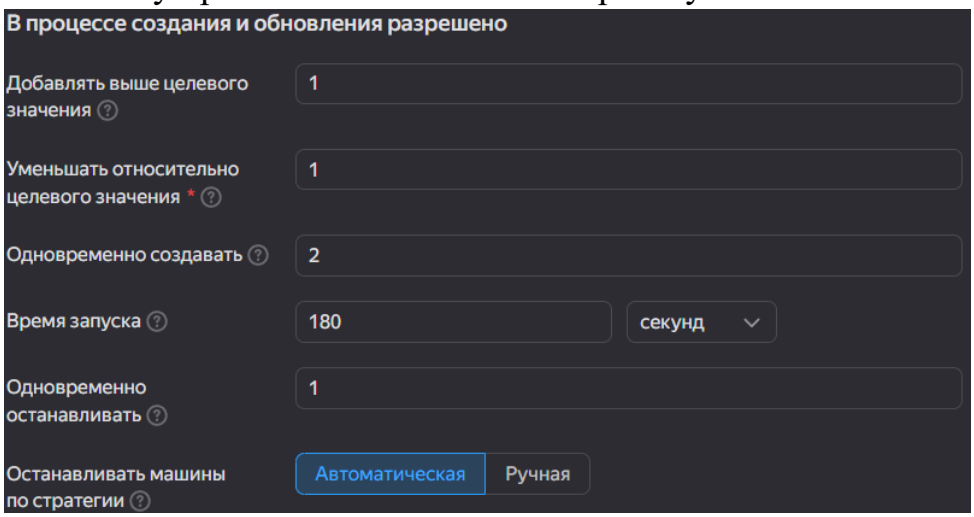
8. Создаю группы виртуальных машин со следующими параметрами:



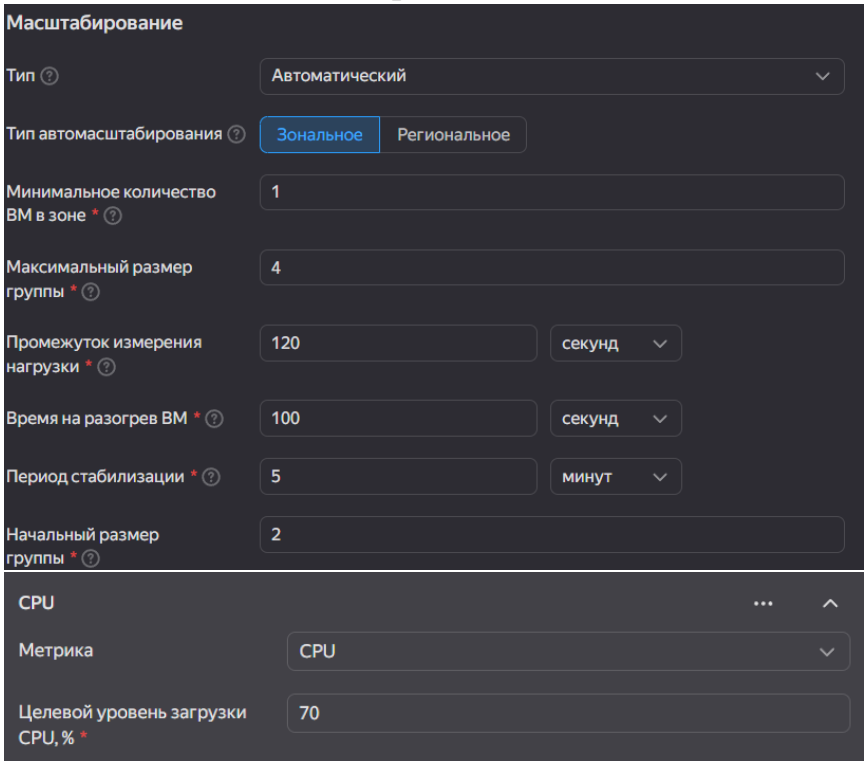
9. Также там необходимо создать шаблон ВМ, до этого я отдельно создавала каждую ВМ, при этом способе, он создаст автоматически по заданному шаблону:



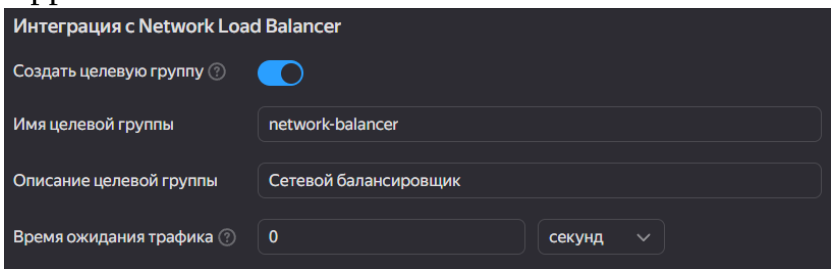
10. Добавляю условия для своей тестовой группы, выбираю минимальные значения. Время запуска указано с расчетом старта ВМ и загрузки ОС, значения ниже могу привести к постоянном перезапуске ВМ



11. Значения для масштабирования:



12. Создаю балансировщик нагрузки, на выбор можно либо Network Balancer, либо Application balancer.



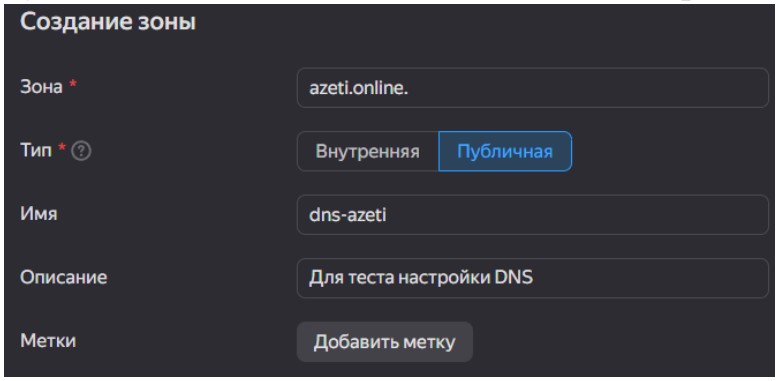
13. Как настройка выглядела бы, если использовать CLI:

```
yc compute instance-group create\
  --folder-id b1gpmga0r88fkk7tbv6m\
  --file <( cat << 'CLOUD_CONSOLE_INSTANCE_GROUP_SPEC'
folderId: b1gpmga0r88fkk7tbv6m
name: vm-group
description: Группа ВМ для настройки балансера
serviceAccountId: aje6r3nh2s52gd18aujj
deletionProtection: false
allocationPolicy:
  zones:
    - zoneId: ru-central1-d
deployPolicy:
  strategy: PROACTIVE
  maxCreating: '2'
  maxDeleting: '1'
  maxUnavailable: '1'
  maxExpansion: '1'
  startupDuration:
```

```
    seconds: '180'
instanceTemplate:
  networkInterfaceSpecs:
    - networkId: enpjkprmr0okmfscbv65
      subnetIds:
        - fl8vnvq6cui671499bvo
      securityGroupIds:
        - enptlj8qiuu6ee770p3n
      primaryV4AddressSpec:
        oneToOneNatSpec:
          ipVersion: IPV4
        dnsRecordSpecs: []
  labels: {}
  name: ''
  description: ''
  resourcesSpec:
    cores: '2'
    memory: '2147483648'
    coreFraction: '100'
    gpus: '0'
  schedulingPolicy:
    preemptible: true
  platformId: standard-v3
  bootDiskSpec:
    diskSpec:
      name: disk-ubuntu-24-04-lts-1786702496501
      typeId: network-ssd
      size: '10737418240'
      imageId: fd8udtumfhprimadiega
    mode: READ_WRITE
  secondaryDiskSpecs: []
  filesystemSpecs: []
  serviceAccountId: aje6r3nh2s52gd18aujj
  metadata:
    user-data: |-
      #cloud-config
      datasource:
       Ec2:
        strict_id: false
      ssh_pwauth: no
      users:
      - name: anastasia
        sudo: ALL=(ALL) NOPASSWD:ALL
        shell: /bin/bash
```

```
        ssh_authorized_keys:
        - ssh-rsa
[key]
    ssh-keys: anastasia:ssh-rsa
[key]
    private_ui_access-method: ssh-key
scalePolicy:
  autoScale:
    autoScaleType: ZONAL
    minZoneSize: '1'
    maxSize: '4'
    measurementDuration:
      seconds: '120'
    warmupDuration:
      seconds: '100'
    stabilizationDuration:
      seconds: '300'
    initialSize: '2'
    cpuUtilizationRule:
      utilizationTarget: 70
loadBalancerSpec:
  targetGroupSpec:
    name: network-balancer
    description: Сетевой балансировщик
variables: []
CLOUD_CONSOLE_INSTANCE_GROUP_SPEC
)
```

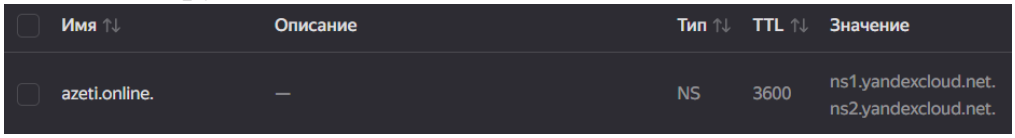
14. Перехожу к настройке DNS. Ранее на рег ру я приобрела доменное имя (azeti.online) его попробую привязать к сервису Яндекс. Тип выбираю публичный, чтобы его было видно из интернета



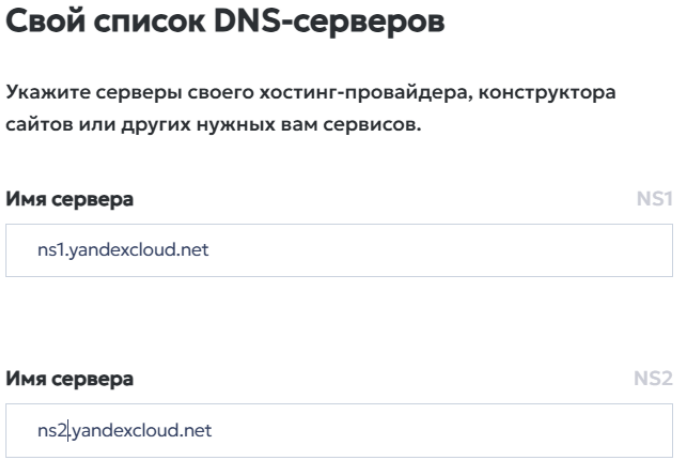
15. Как создание выглядело бы с помощью CLI:

```
yc dns zone create \
  --name dns-azeti \
  --description Для теста настройки DNS \
  --zone azeti.online. \
  --public-visibility=true
```

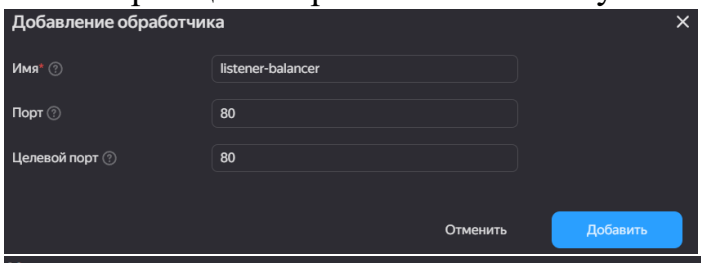
16. Для привязки моего ДНС, мне необходимо перейти в созданный только что ДНС. Копирую два значения от яндекса:

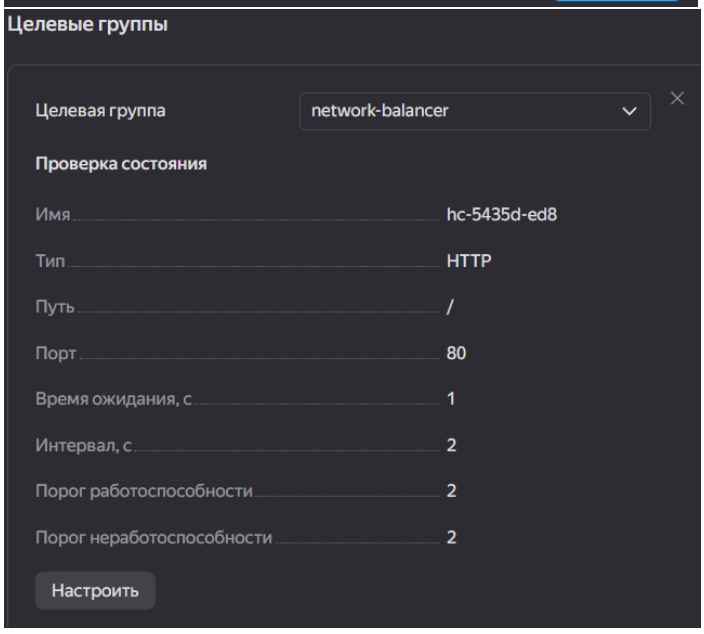


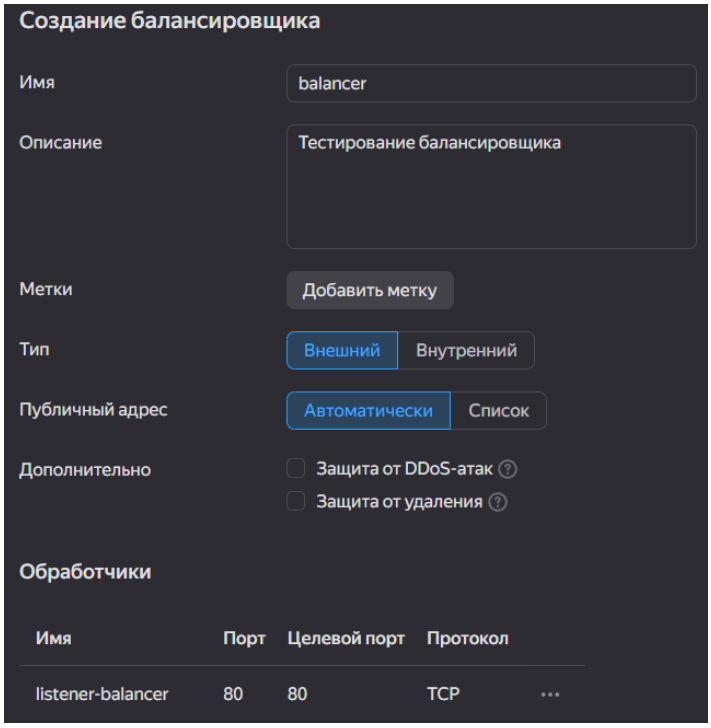
Перехожу на рег ру и выбираю пункт с указанием собственных DNS серверов и туда вписываю значения выше. Обновление записей может занять до 24 часов, и либо сайт будет ссылаться на доменное имя рег ру, либо будет недоступен:



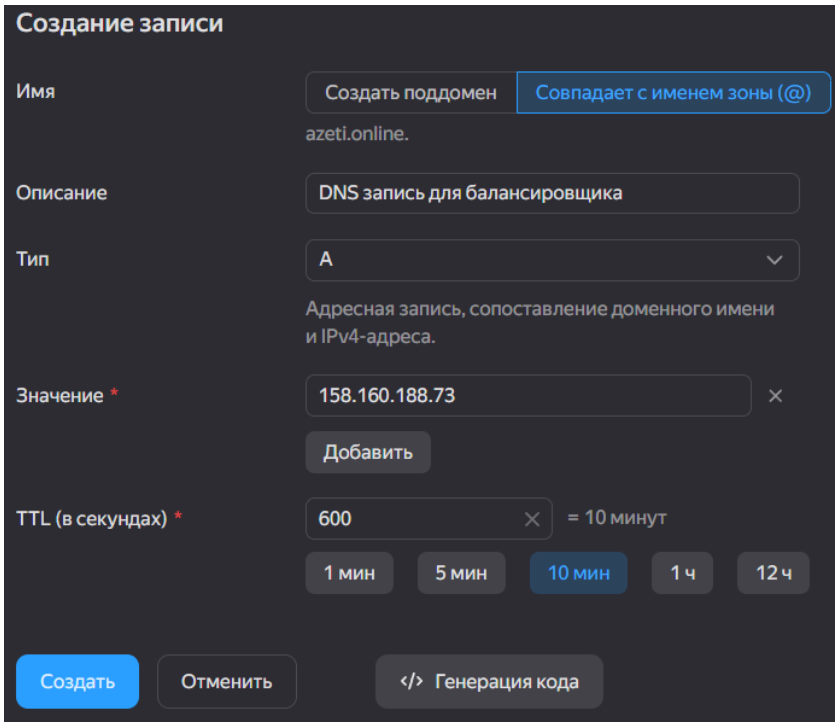
17. Теперь необходимо настроить новую запись и привязать к ней айпи адрес балансировщика, тогда при автомасштабировании и замене ВМ адрес сайта не меняется — трафик распределяет балансировщик, а целевая группа обновляется автоматически.
18. Как оказалось, это была создана целевая группа, а не сам балансировщик, тогда непонятно для чего там была автонастройка балансировщика. Итак, создаем балансировщик и привязываем к нему наше целевую группу:



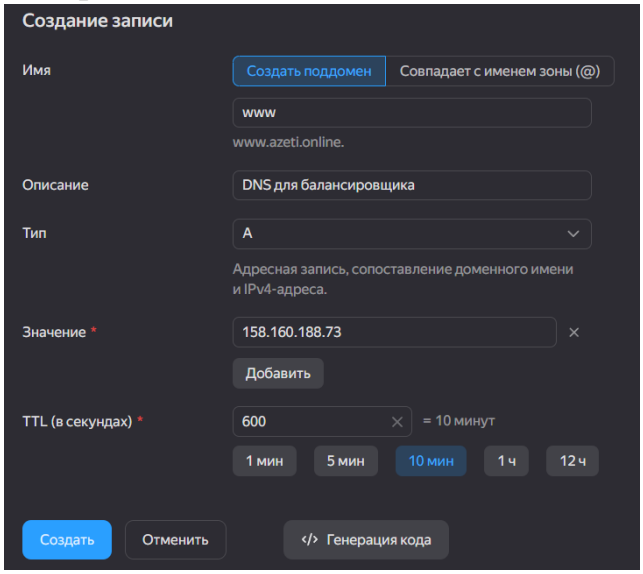




19. Как бы настройка выглядела с помощью CLI:
    yc load-balancer network-load-balancer create \
      --name balancer \
      --description Тестирование балансировщика \
      --type external \
      --listener name=listener-balancer,port=80,target-port=80,protocol=tcp,external-ip-version=ipv4 \
      --target-group target-group-id=enpnrmc6thv8oh07ag2s,healthcheck-name=hc-5435d-ed8,healthcheck-interval=2,healthcheck-timeout=1,healthcheck-unhealthythreshold=2,healthcheck-healthythreshold=2,healthcheck-http-port=80,healthcheck-http-path=/
20. Теперь добавим запись DNS и укажем там айпи адрес балансировщика 158.160.188.73



21. Теперь также добавляем запись www



22. Теперь проверим через командную строку с помощью команд должен вывести айпи адрес нашего балансировщика:
    - dig +short azeti.online @ns1.yandexcloud.net
    - dig +short www.azeti.online @ns1.yandexcloud.net

```
anastasia@cl1cclk1t73mvoc2gu73-efyz:~$ dig +short azeti.online @ns1.yandexcloud.net
158.160.188.73
anastasia@cl1cclk1t73mvoc2gu73-efyz:~$ dig +short www.azeti.online @ns1.yandexcloud.net
158.160.188.73
```

23. Я показала, как делегировать, но из-за долгого делегирования показать результат, что оно применилось не могу, но настройка верна.

```
anastasia@cl1cclk1t73mvoc2gu73-efyz:/etc/nginx$ yc dns zone list-records dns-azeti | grep '158.160.188.73'
| azeti.online.     | 600  | A    | 158.160.188.73              | DNS запись для балансировщика |
| www.azeti.online. | 600  | A    | 158.160.188.73              | DNS для балансировщика        |
```

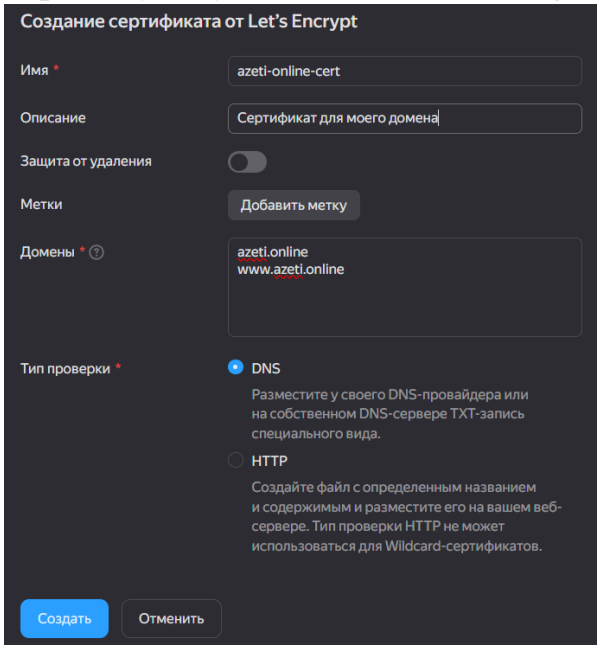
24. Так нужно поднять сервер nginx чтобы провести проверку, который будет на 80 порту, так как балансировщик слушает этот порт. Итак теперь при прогоне curl с адресом балансировщика выдает положительный результат:

```
anastasia@cl1cclk1t73mvoc2gu73-efyz:/etc/nginx$ curl -I http://158.160.188.73
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Fri, 14 Aug 2026 13:07:28 GMT
Content-Type: text/html
Content-Length: 615
Last-Modified: Fri, 14 Aug 2026 13:02:11 GMT
Connection: keep-alive
ETag: "6a7f11d3-267"
Accept-Ranges: bytes
```

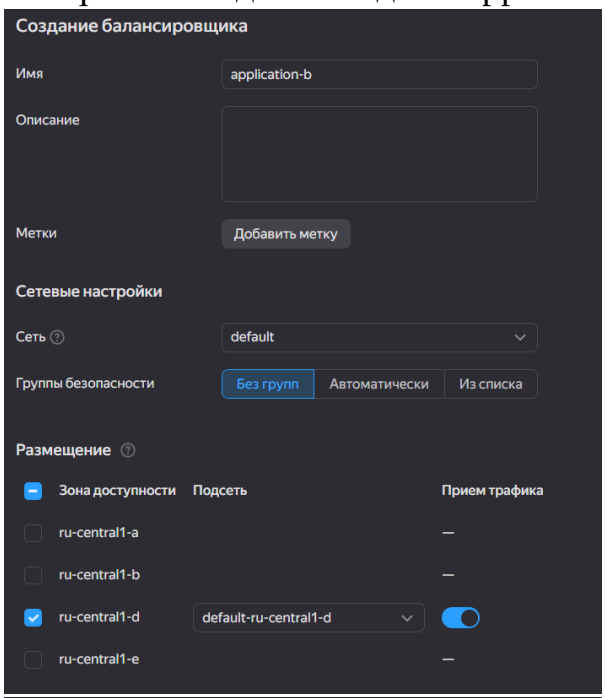
25. Так как делегирование займет примерно сутки, протестируем работа в обход с помощью команды curl -I --max-time 10 --resolve azeti.online:80:158.160.188.73 http://azeti.online. Как видно тест успешен

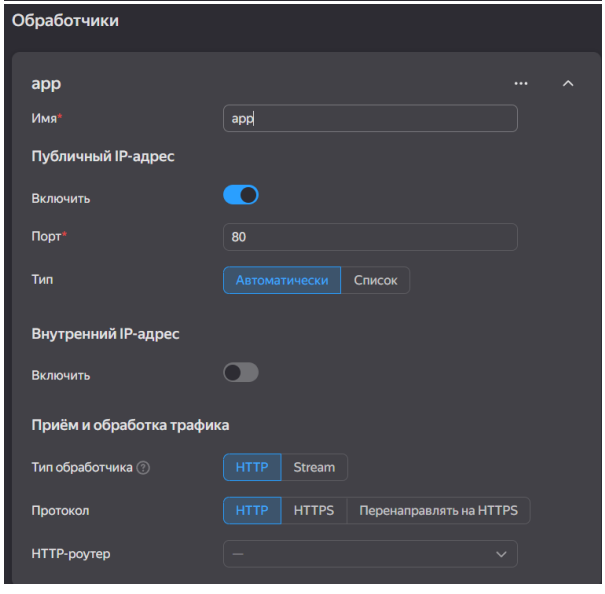
```
anastasia@cl1cclk1t73mvoc2gu73-efyz:/etc/nginx$ curl -I --max-time 10 --resolve azeti.online:80:158.160.188.73 http://azeti.online
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Fri, 14 Aug 2026 13:09:57 GMT
Content-Type: text/html
Content-Length: 615
Last-Modified: Fri, 14 Aug 2026 13:02:11 GMT
Connection: keep-alive
ETag: "6a7f11d3-267"
Accept-Ranges: bytes
```

26. Теперь перехожу к созданию SSL сертификата для своего домена. Для этого перехожу в пункт Certificate Manager – Сертификат от Let’s Encrypt:

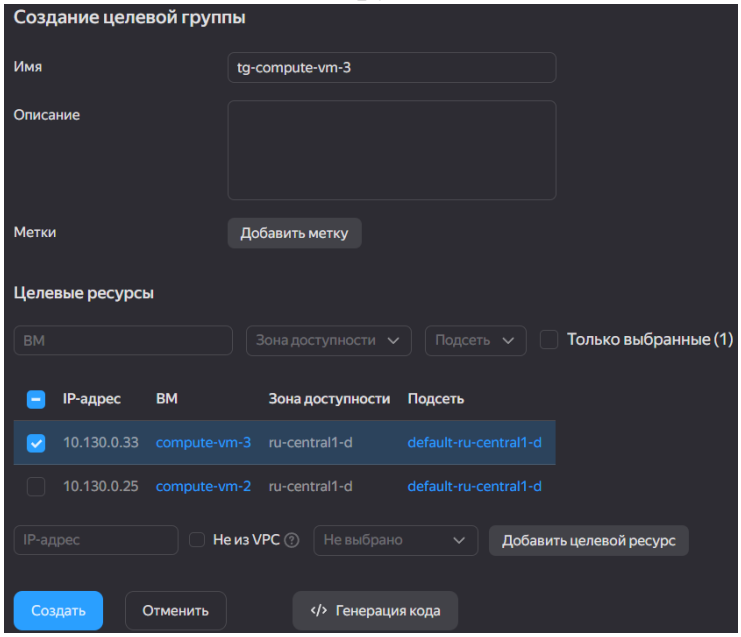


27. Теперь необходимо создать Application Load Balancer:





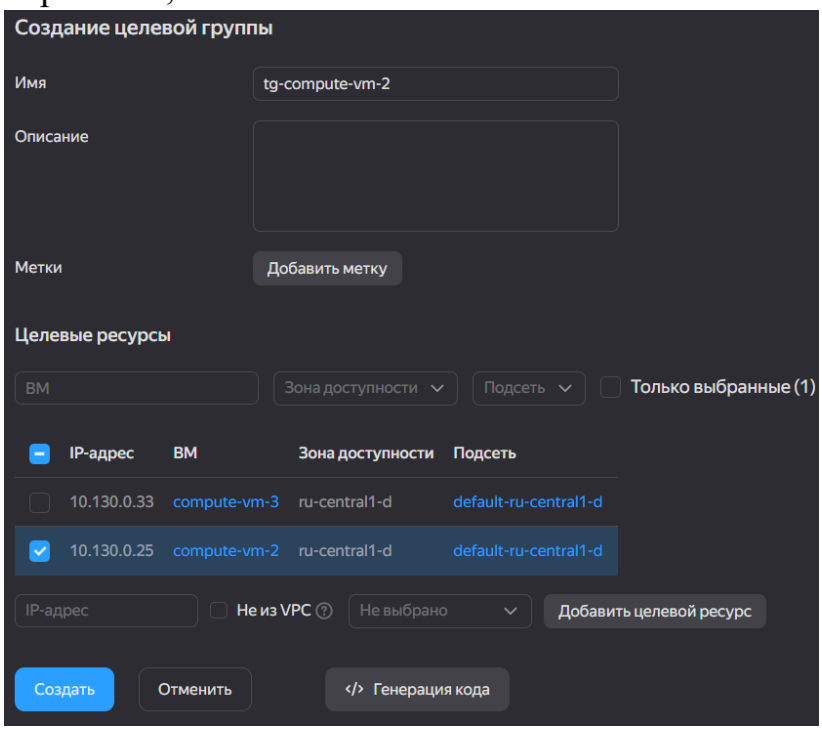
28. После создания теперь необходимо создать группу бэкендов. А для этого дополнительную ВМ с такими же характеристиками. Теперь переходим в ALB и создаем две целевые группы



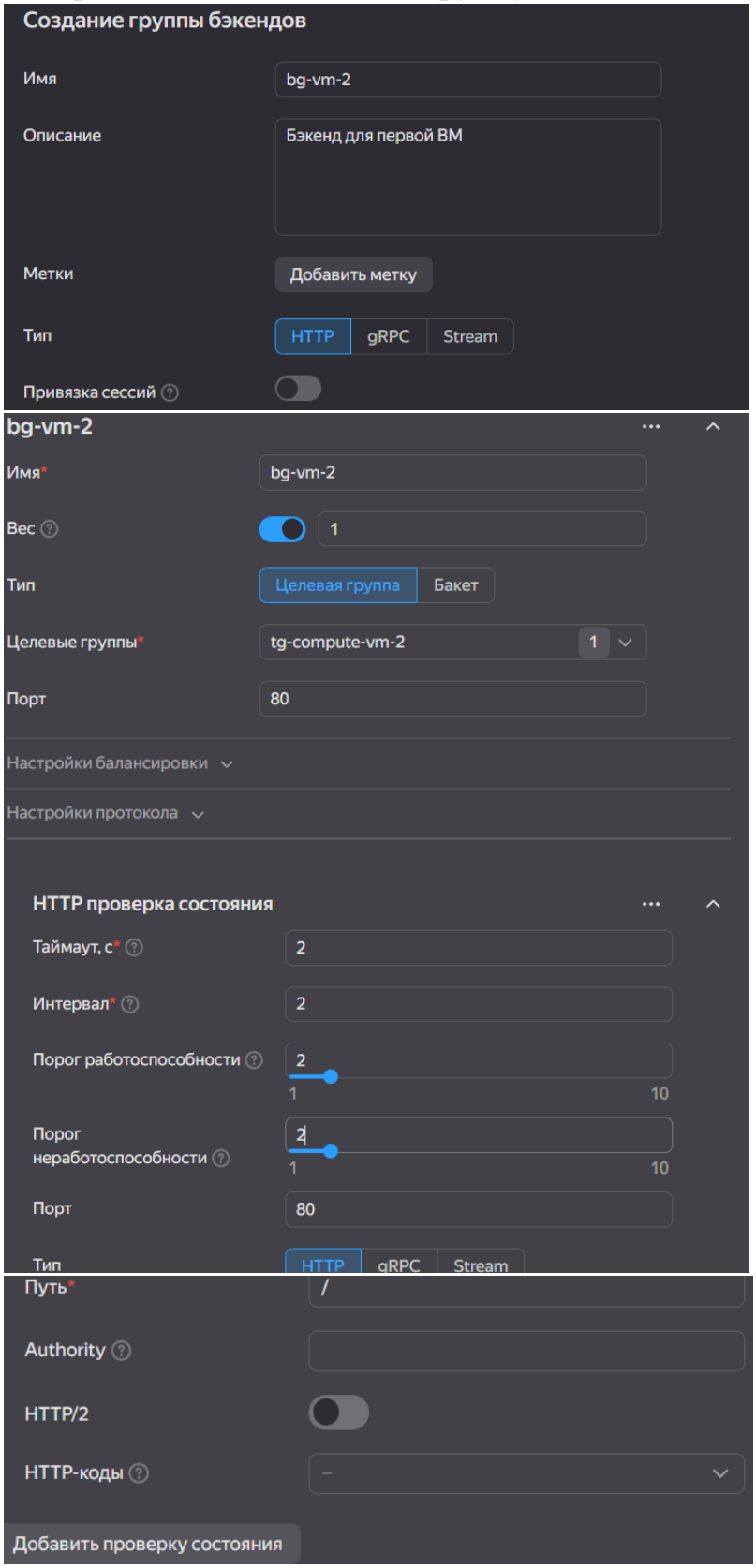
29. Захожу на созданную ВМ и выполняю следующие действия (но сначала устанавливаем nginx):
    - sudo mkdir -p /var/www/html/api
    - echo "v2" | sudo tee /var/www/html/api/version
    - curl -s http://localhost/api/version

```
puppy@compute-vm-3:~$ curl -s http://localhost/api/version
v2
```

30. Также добавляю группу для нашей ВМ, по аналогии. И те же команды выше на первой вм, только запись v1:



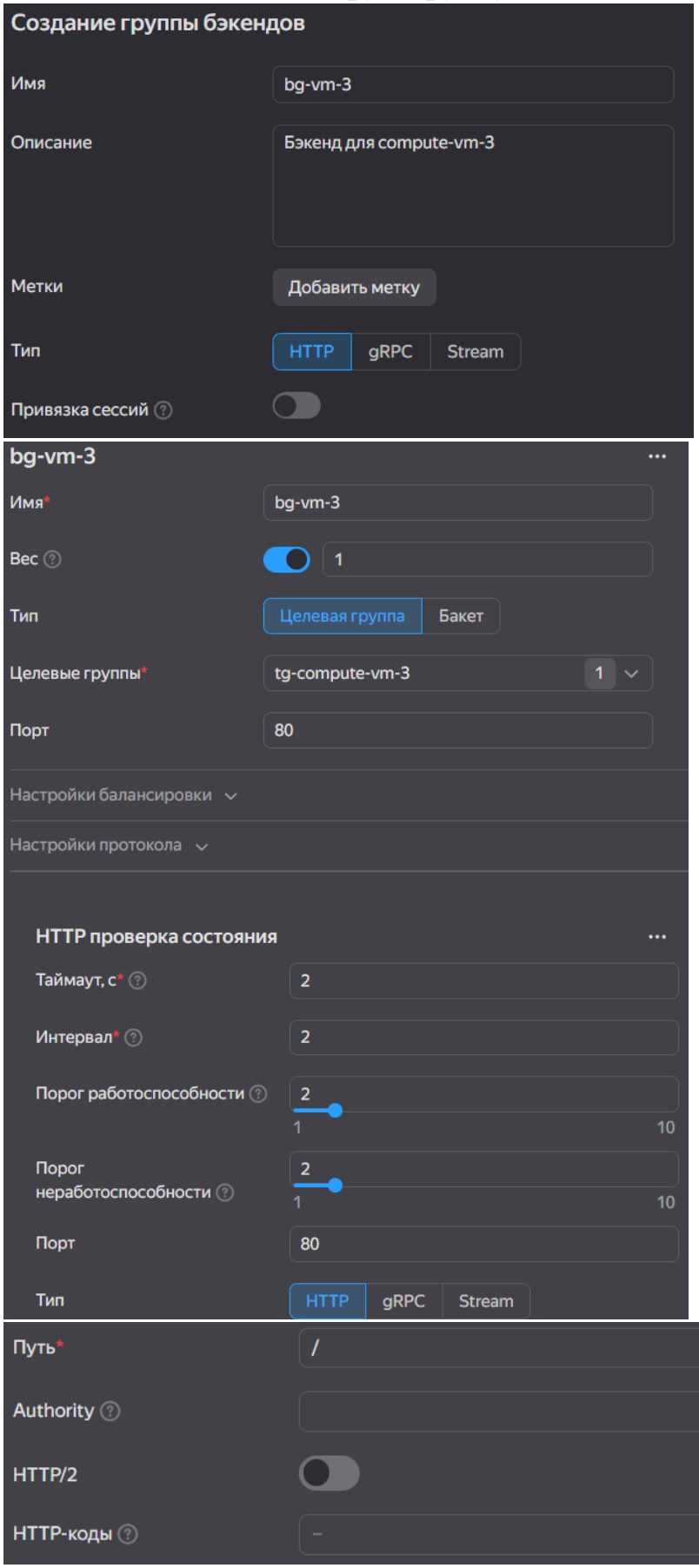
31. Теперь необходимо создать группу бэкэндов. Создаем сначала для первой ВМ:



32. Как бы это выглядело через CLI:

```
yc application-load-balancer backend-group create \
  --name bg-vm-2 \
  --description Бэкенд для первой ВМ
```

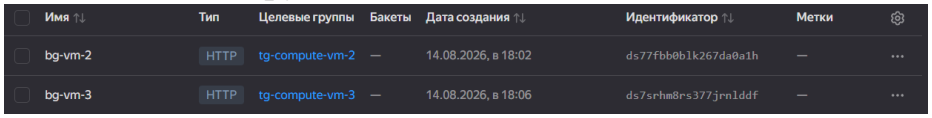
33. По аналогии создаем вторую группу бэкенда



34. Как бы это выглядело через CLI:

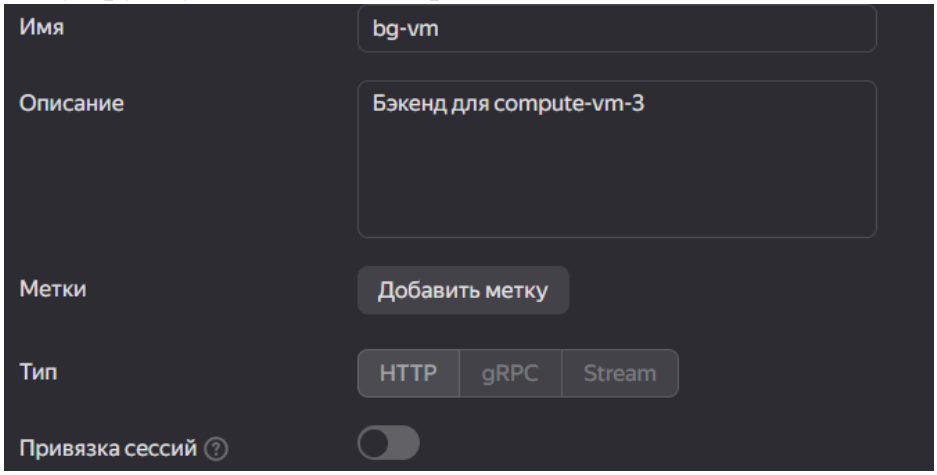
```
yc application-load-balancer backend-group create \
  --name bg-vm-3 \
  --description Бэкенд для compute-vm-3
```

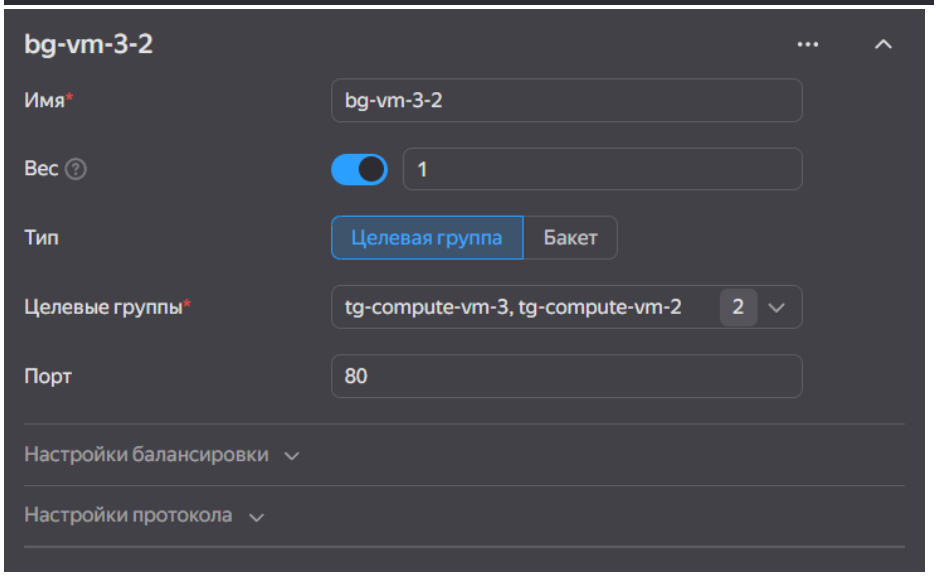
35. Вот созданные группы:



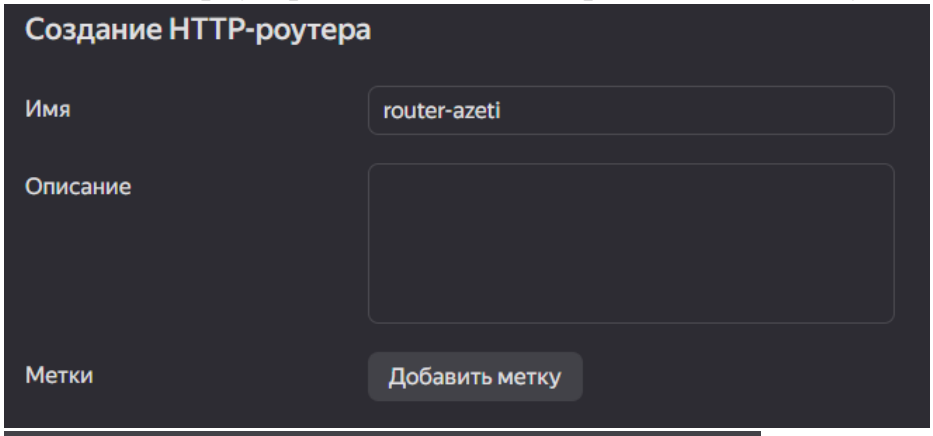
Имя	Тип	Целевые группы	Бакеты	Дата создания	Идентификатор	Метки
bg-vm-2	HTTP	tg-compute-vm-2	—	14.08.2026, в 18:02	ds77fbb0blk267da0a1h	—
bg-vm-3	HTTP	tg-compute-vm-3	—	14.08.2026, в 18:06	ds7srhm8rs377jrnlddf	—

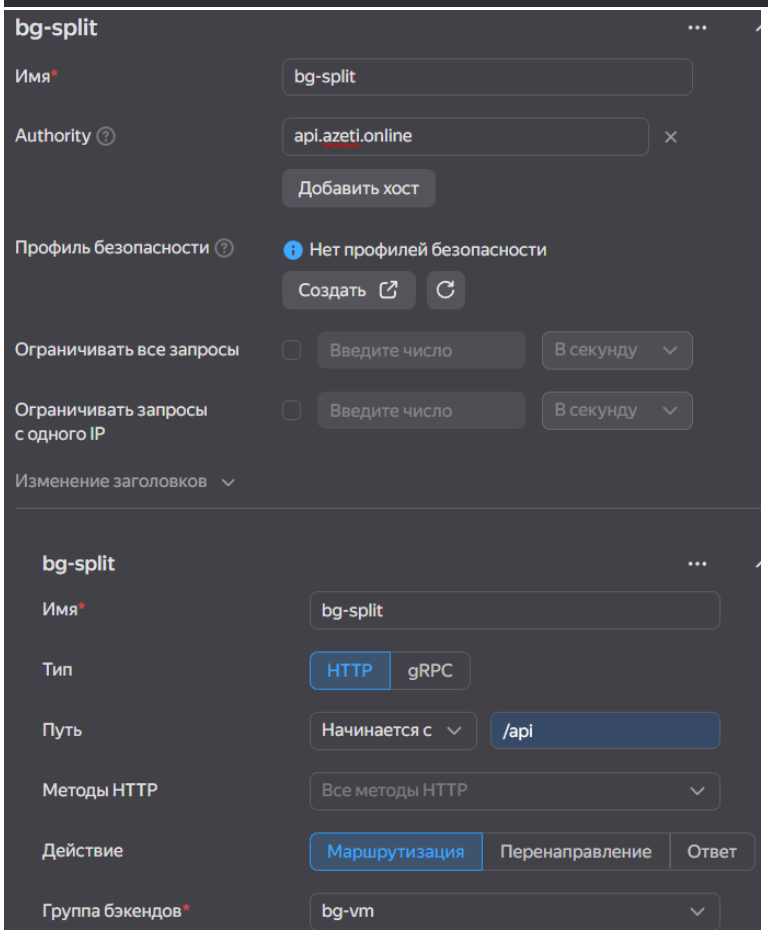
36. Теперь переходим к созданию HTTP-роутер, пришлось объединить две группы в одну группу бэкенда. Настройки остались без изменений:





37. Создаю сам роутер. Все что не на скриншоте, все по умолчанию



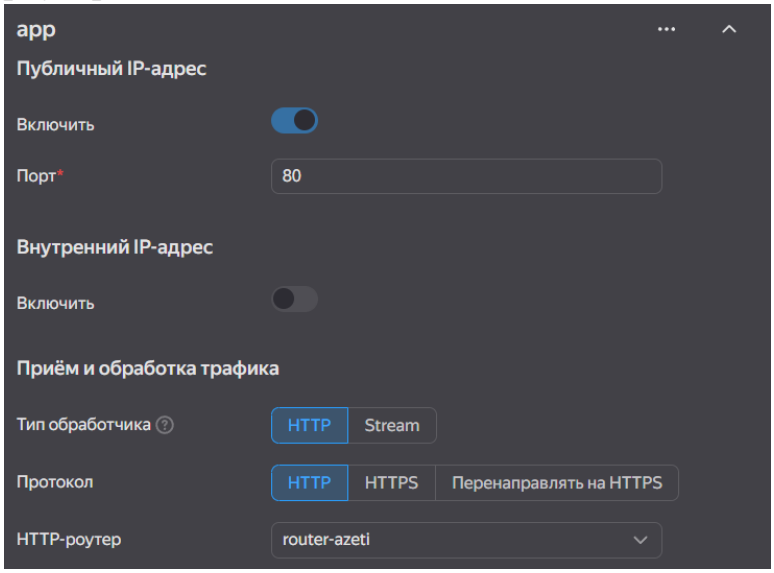


38. Как бы выглядело через CLI:

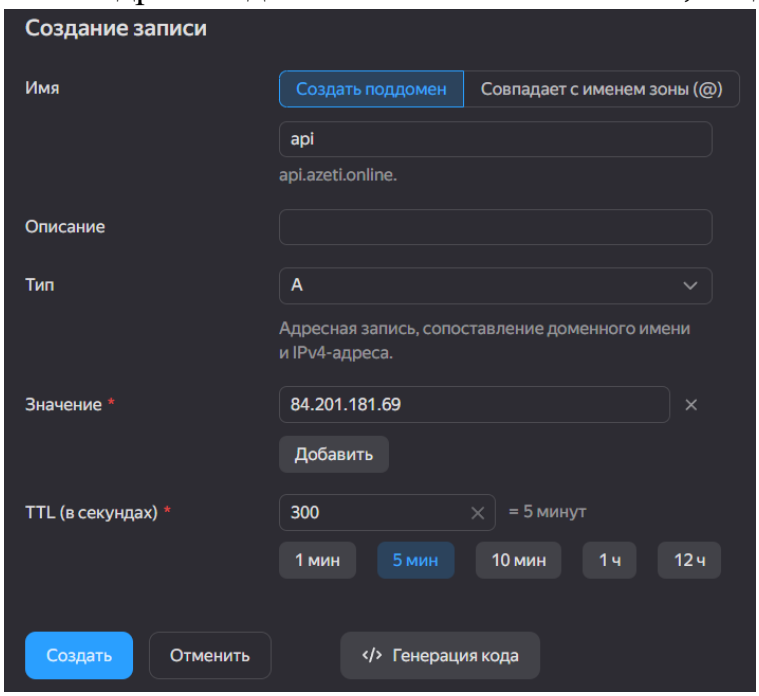
```
yc application-load-balancer http-router create \
  --name router-azeti

yc application-load-balancer virtual-host create \
  --name bg-split \
  --http-router-name router-azeti \
  --authority api.azeti.online
```

39. Теперь захожу в редактирование созданного ALB ранее и добавляю созданный роутер:



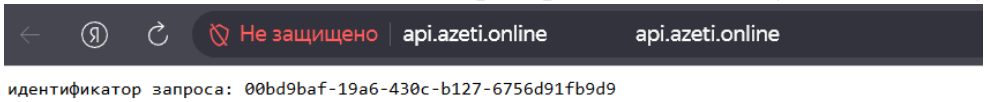
40. Айпи адрес созданного ALB 84.201.181.69, создаем для него DNS запись:



41. Теперь проверяем заходим на compute-vm-2 и вводим следующее. Как видно он начал отдавать айпи адрес нашего роутера:

```
anastasia@cl1cc1k1t73mvoc2gu73-efyz:~$ dig +short api.azeti.online @ns1.yandexcloud.net
84.201.181.69
```

42. Если ввести в браузере адрес http://api.azeti.online/api/version, то скачается файл с версией (если обновить несколько раз, то можно увидеть что он отдает не только v1, но и v2). А если ввести http://api.azeti.online/ увидим следующее:

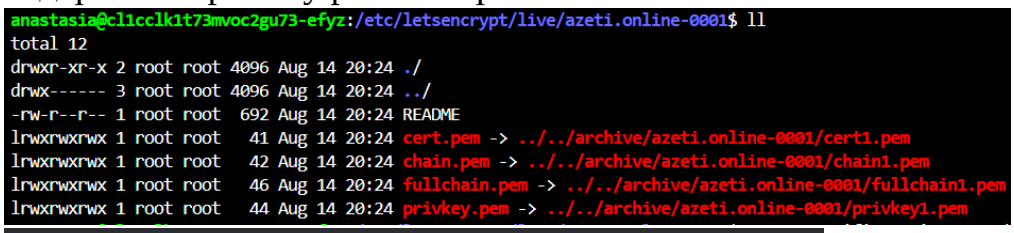


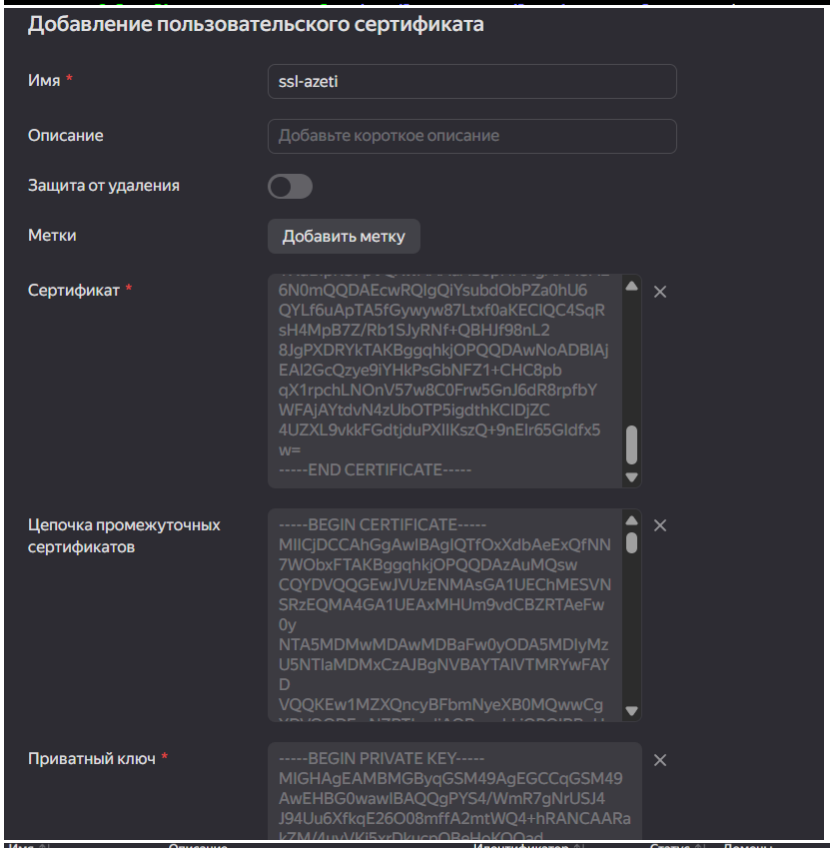
идентификатор запроса: 00bd9baf-19a6-430c-b127-6756d91fb9d9

43. Теперь добавлю api.azeti.online в созданный сертификат azeti-online-cert. Почему-то он не может пройти валидацию, хотя делегирование из рег ру уже прошло успешно. Попробую выпустить сертификат на самой ВМ с помощью certbot. После установки прогняем команду

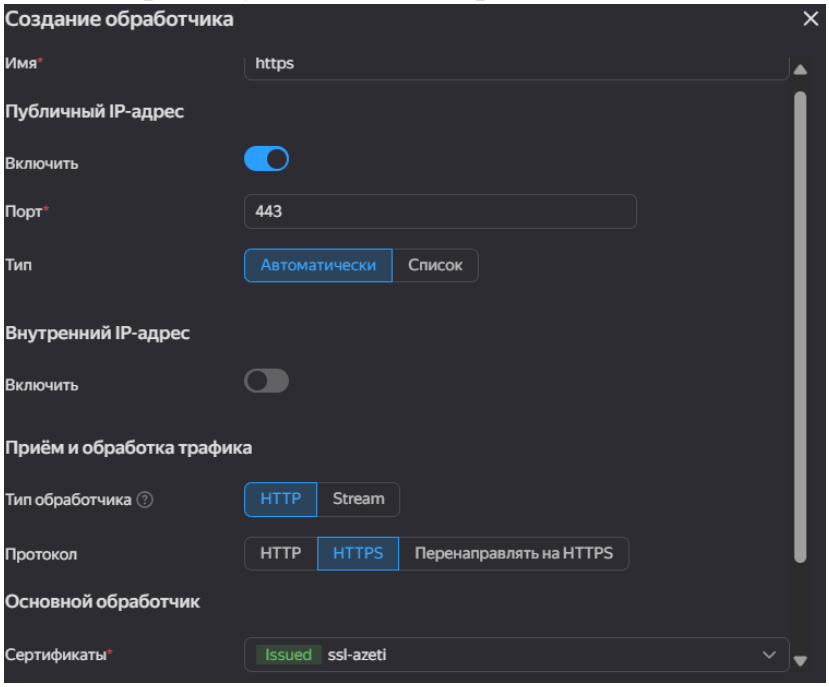
    sudo certbot certonly --webroot -w /var/www/html \
      -d azeti.online -d www.azeti.online -d api.azeti.online \

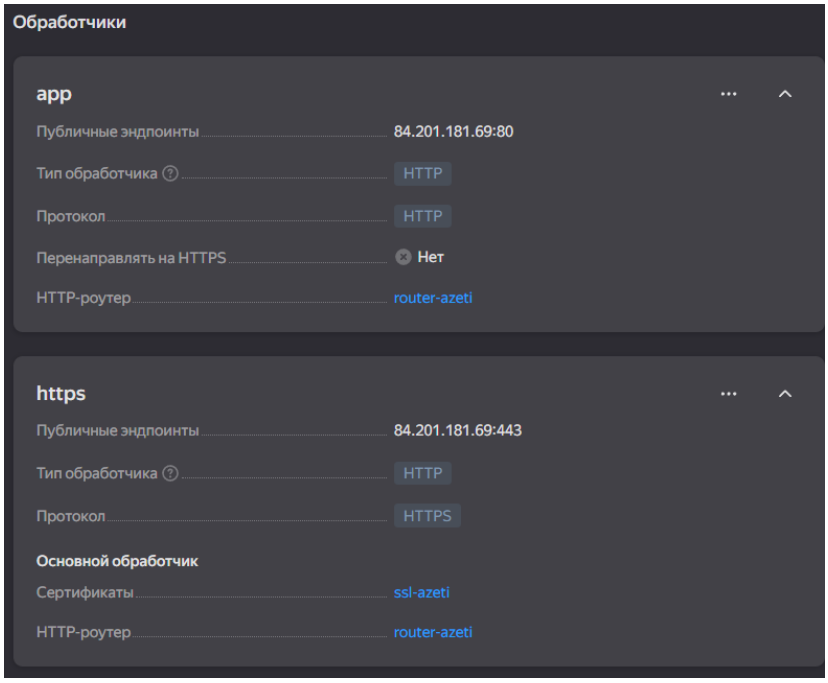
44. Теперь необходимо добавить сертификат в yandex certificate manager, но вместо автовыпуска серта, выбираю загрузить пользовательский. В после “Сертификат” вставляем содержимое из cert.pem. В цепочку промежуточных вставляем содержимое chain.pem. Соответственно в строку «Приватный ключ» вставить содержимое privkey.pem. Сохраняем





45. Далее перехожу в созданный ранее ALB и создаю еще один обработчик:

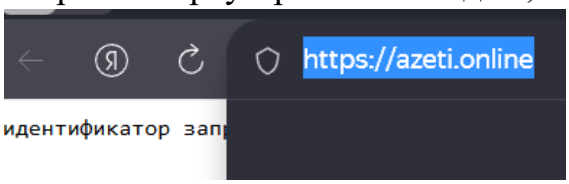




46. Далее перехожу в Cloud DNS и меняю айпи адреса у azeti.online и www.azeti.online, чтобы они были одинаковы

Имя	Описание	Тип	TTL	Значение	Владелец
api.azeti.online.	—	A	300	84.201.181.69	—
azeti.online.	DNS запись для балансировщика	A	600	84.201.181.69	—
www.azeti.online.	DNS для балансировщика	A	600	84.201.181.69	—

47. Теперь тестируем в командной строке вводя поочередно команды в обход кэша DNS (с https://azeti.online), так как он держит в кэше старый айпи адрес от рег ру. Но api уже смотрит правильно:
    - curl -I --resolve azeti.online:443:84.201.181.69 https://azeti.online. Но если открыть в браузере то как видим, сайт открывается и имеет ssl сертификат



    - curl -I https://www.azeti.online

```
anastasia@cl1cclk1t73mvoc2gu73-efyz:~$ curl -I https://www.azeti.online
HTTP/2 404
content-length: 50
content-type: text/plain
date: Fri, 14 Aug 2026 21:09:06 GMT
server: ycalb
```

    - curl -I https://api.azeti.online/api/version

```
anastasia@cl1cclk1t73mvoc2gu73-efyz:~$ curl -I https://api.azeti.online/api/version
HTTP/2 200
server: ycalb
date: Fri, 14 Aug 2026 21:05:34 GMT
content-type: application/octet-stream
content-length: 3
last-modified: Fri, 14 Aug 2026 14:53:44 GMT
etag: "6a7f2bf8-3"
accept-ranges: bytes
```